

Eliminating Timing Channels in Microkernels: Integrating Time Protection Mechanisms into FreeRTOS

Simon Vinding Brodersen
Department of Computer Science
University of Copenhagen
Copenhagen, Denmark
rzn875@alumni.ku.dk

Abstract—Timing channels present a significant security threat in modern computing, allowing unauthorized information flow through observable variations in execution time. Research has been done on creating secure microkernels designed to mitigate these channels from the ground up, while other research focuses on closing timing channels at the user level. It remains an open challenge to gather these findings for a more general and mainstream OS, and this paper investigates the feasibility of integrating time protection mechanisms into the widely used FreeRTOS kernel.

FreeRTOS provides a default priority-based preemptive scheduler, which is inherently vulnerable to timing channels. To address this, this work implements a domain-level scheduler that partitions execution into fixed time slices, ensuring that the execution of tasks in one domain does not affect the timing observed by another. Key to this implementation is the temporal fence (`fence.t`) instruction during domain switches to clear shared microarchitectural states. Further, constant-time domain creation mechanisms are introduced, allowing dynamic resource reconfiguration without introducing new timing channels.

The implementation was evaluated using QEMU system emulation. The results demonstrate that the modified kernel achieves constant-time domain scheduling with negligible variance with regard to instruction counting. While evaluation on true hardware remains undone, this work provides a foundation towards bringing time protection to mainstream embedded systems.

I. INTRODUCTION

II. BACKGROUND

- A. *Side-Channels and Covert Channels*
- B. *OS-Level Timing Channels*
- C. *Spatial and Temporal Isolation*
- D. *The `fence.t` Instruction*
- E. *FreeRTOS and FreeRTOS-MPU*

III. RELATED WORK

- A. *User-Level Mitigations*
- B. *seL4 Time Protection*
- C. *S3K*
- D. *Research Gap*

IV. DESIGN

- A. *FreeRTOS Scheduler Vulnerability*
- B. *Approaches to Closing Scheduling-Based Channels*
- C. *Domain-Level Scheduler Design*
- D. *Constant-Time Domain Creation*
- E. *Cross-Domain Communication*

V. IMPLEMENTATION

- A. *API Extensions*
- B. *Configuration Flags*
- C. *Domain Switching*
- D. *Per-Domain Scheduling State*
- E. *Cross-Domain Queues*

VI. EVALUATION

- A. *Evaluation Platform and Limitations*
- B. *Blinky Demo: Two Domains*
- C. *Multi-Domain Demo: Four Domains*
- D. *Architectural Complexity and Throughput Cost*

VII. DISCUSSION AND FUTURE WORK

- A. *Hardware Verification*
- B. *System-Call Audit*
- C. *Remaining Timing Channels*
- D. *Lighter-Weight Alternatives*

VIII. CONCLUSION